

Quy trình Render 3D Gaussian Splatting và Cải tiến FastGS (FastGS-lite)

Bản rút gọn 45 phút — Từ nền tảng đến Adaptive Density Control

Digital Twin GS

Ngày 23 tháng 9 năm 2026

- 1. Nền tảng: 3D Gaussian & Spherical Harmonics (lược gọn)
- 2. Chiếu phối cảnh & Rasterizer khả vi
- 3. Hàm mất mát, Gradient & Optimizer
- 4. **Trọng tâm:** Adaptive Density Control – cải tiến đề xuất
- 5. Ba đòn bẩy tăng tốc của FastGS
- 6. Kết quả thực nghiệm: FastGS-lite trên Colab T4
- 7. Quan sát, giới hạn & kết luận

Phần nền tảng toán học sẽ lược gọn tối đa, dành thời gian cho đóng góp chính và số liệu thực nghiệm.

Động lực: vì sao cần một biểu diễn cảnh mới?

- NeRF: biểu diễn **implicit** bằng MLP, chất lượng cao nhưng render chậm (giây/khung hình) do ray-marching.
- 3D Gaussian Splatting (3DGS): biểu diễn **explicit** bằng hàng trăm nghìn Gaussian mờ, render bằng **rasterization khả vi**.
- Kết quả: hàng chục–hàng trăm FPS, gần thời gian thực.

Tiêu chí	NeRF	3D Gaussian Splatting
Biểu diễn	Implicit (MLP)	Explicit (tập Gaussian)
Render	Ray-marching, chậm	Rasterization, nhanh
Tốc độ	~giây/khung hình	Hàng chục–hàng trăm FPS
Huấn luyện	Backprop qua MLP	Backprop qua tham số Gaussian

3D Gaussian là gì?

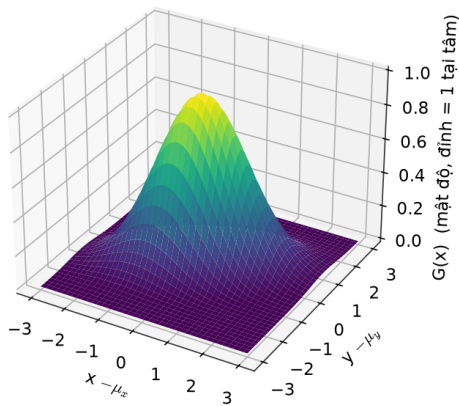
Hàm mật độ Gaussian 3D:

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

Bốn nhóm tham số học được:

- Vị trí $\mu \in \mathbb{R}^3$
- Hiệp phương sai Σ (hình dạng, hướng)
- Độ mờ (opacity) α
- Màu qua hệ số Spherical Harmonics

$$G_i(x) = \exp\left(-\frac{1}{2}(x - \mu_i)^T \Sigma_i^{-1}(x - \mu_i)\right)$$



Hình: Quả chuông mật độ Gaussian 3D

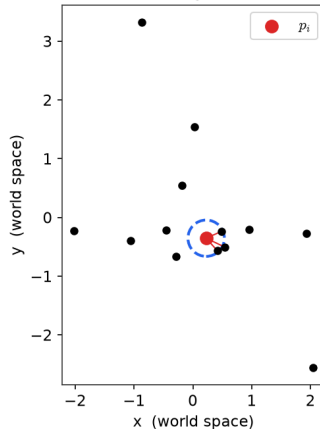
Tham số hoá hiệp phương sai

- Không học trực tiếp Σ để đảm bảo tính **positive semi-definite** khi tối ưu.
- Thay vào đó học scale $s \in \mathbb{R}^3$ và quaternion xoay $q \in \mathbb{R}^4$:

$$\Sigma = RSS^T R^T$$

với $S = \text{diag}(s)$, $R = R(q)$.

$\tilde{s}_i = \log \sqrt{d_{knn3}^2(p_i)} \cdot \vec{1}_3$
(bán kính = RMS khoảng cách tới 3 láng giềng)

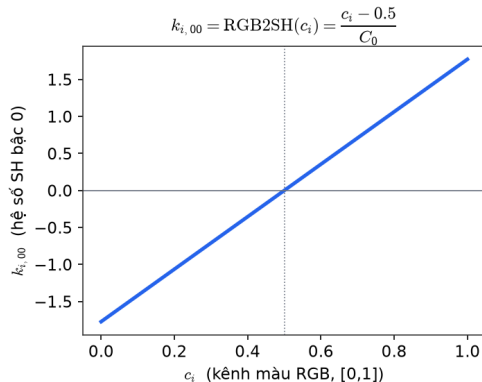


Hình: Khởi tạo scale từ khoảng cách lân cận

Màu sắc qua Spherical Harmonics (SH)

- Mỗi Gaussian không lưu 1 màu RGB cố định, mà lưu hệ số **SH**.
- Biểu diễn màu **phụ thuộc góc nhìn** (view-dependent), đặc tả hiệu ứng phản xạ/specular.
- Hệ số SH bậc 0 (DC) khởi tạo trực tiếp từ màu RGB quan sát được:

$$f_{dc} = \text{RGB2SH}(\text{color})$$

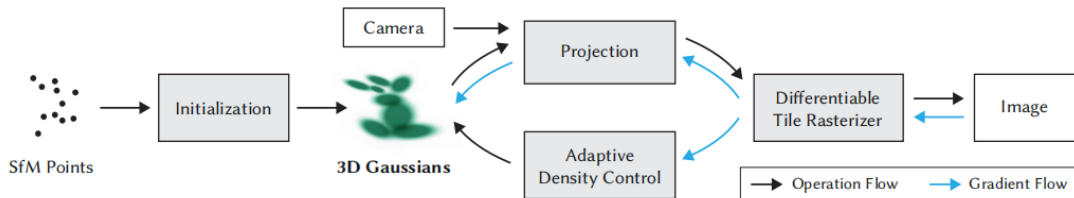


Hình: Chuyển đổi RGB $\rightarrow$ hệ số SH bậc 0

(Phần suy diễn toán học đầy đủ của SH được trình bày chi tiết trong bản đầy đủ, không đọc trong bản 45 phút.)

Tổng quan pipeline render

- World $\rightarrow$ Camera: biến đổi hệ toạ độ bằng view transform (R, t)
- Projection: chiếu các Gaussian 3D xuống mặt phẳng ảnh 2D
- Tiling + Sorting: xác định tile ảnh hưởng và sắp theo depth
- Alpha blending tuần tự $\rightarrow$ màu pixel cuối cùng



Hình: Pipeline render 3D Gaussian Splatting

Chiếu hiệp phương sai (EWA splatting)

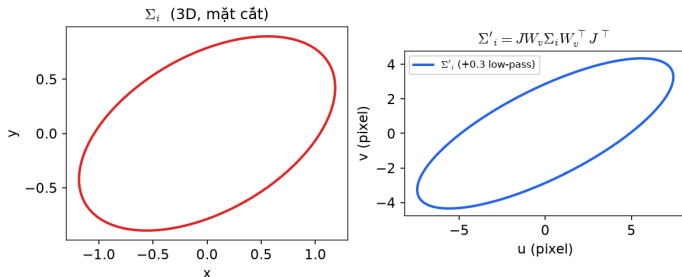
- Phép chiếu phối cảnh phi tuyến $\rightarrow$ xấp xỉ tuyến tính cục bộ bằng Jacobian J tại tâm Gaussian
- Hiệp phương sai 2D sau chiếu:

$$\Sigma' = J W \Sigma W^T J^T$$

trong đó W là phần quay của view matrix

- Kỹ thuật này gọi là **Elliptical Weighted Average (EWA) splatting**

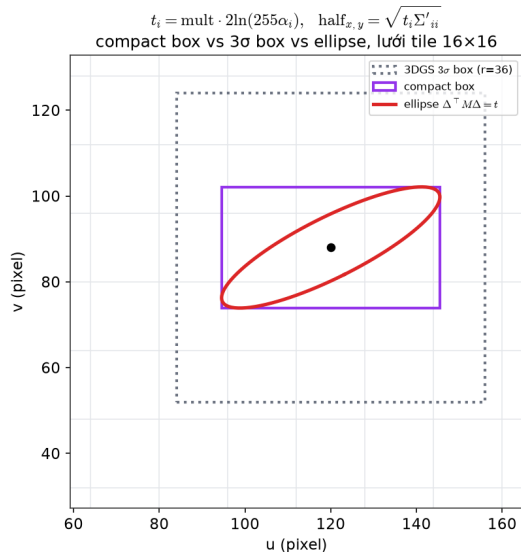
Chiếu covariance: 3D ellipsoid $\rightarrow$ 2D ellipse trên ảnh



Hình: Chiếu hiệp phương sai theo EWA splatting

Compact Box – đòn bẩy tăng tốc #1 của FastGS

- Mặc định: bounding-box 3σ – khá rộng, chạm nhiều tile không cần thiết
- FastGS thay bằng **compact box**: vùng ảnh hưởng 2D nhỏ hơn, điều chỉnh bằng hệ số `--mult`
- Kết quả: giảm đáng kể số tile mỗi Gaussian phải rasterize



Hình: So sánh bounding-box mặc định và compact box

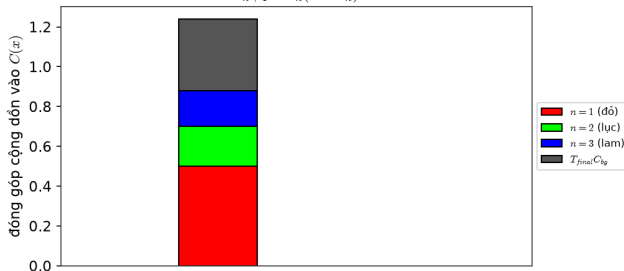
Alpha blending (front-to-back)

- Màu pixel là tổng có trọng số của các Gaussian chồng nhau theo thứ tự độ sâu
- Công thức blend:

$$C = \sum_i c_i \alpha_i T_i, \quad T_i = \prod_{j < i} (1 - \alpha_j)$$

- T_i : độ truyền sáng còn lại (transmittance) trước Gaussian thứ i

$$C(x) = \sum_n c_n \alpha_n T_n + T_{final} C_{bg}$$
$$T_{n+1} = T_n (1 - \alpha_n)$$



Hình: Chồng lớp alpha blending theo độ sâu

- Phương trình render đầy đủ:

$$C(p) = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- Khả vi theo mọi tham số: vị trí μ , hiệp phương sai Σ , độ mờ α , hệ số SH

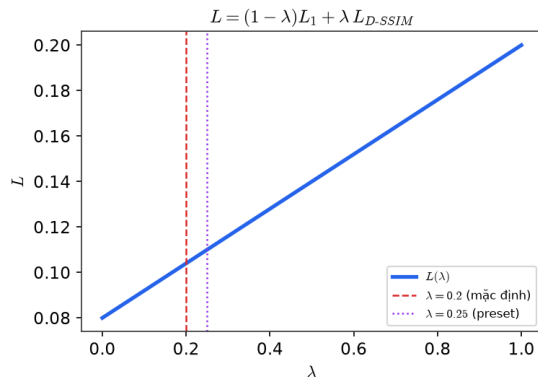
Gaussian 3D → Chiếu 2D → Trường alpha → Blend → Pixel

Hàm mất mát: kết hợp L1 và D-SSIM

- Loss huấn luyện kết hợp hai thành phần bổ trợ nhau:
 - **L1**: sai khác trung bình theo từng pixel.
 - **D-SSIM**: $1 - \text{SSIM}$, phản ánh sai lệch **cấu trúc**.
- Công thức tổng:

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_1 + \lambda (1 - \text{SSIM})$$

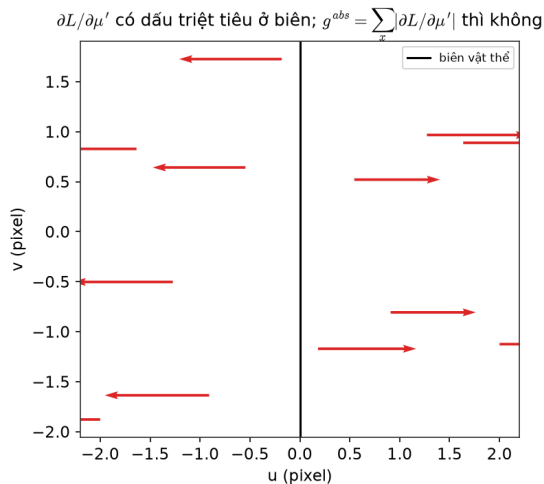
- Hệ số $\lambda = \text{lambda_dssim}$: mặc định 0.2, `fastgs-lite`: 0.25



Hình: Tỷ trọng L1 và D-SSIM trong loss

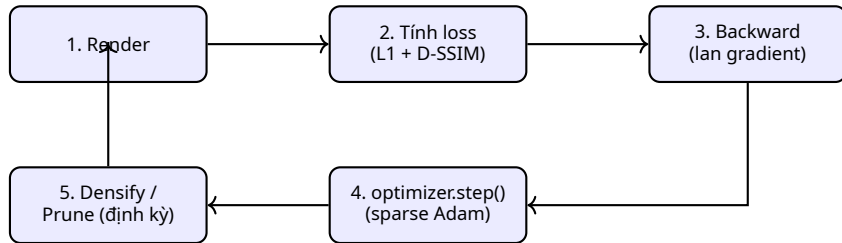
Gradient tập trung tại biên Gaussian

- Gradient view-space lớn nhất xuất hiện ở vùng **biên/rìa** của Gaussian – nơi ảnh render lệch nhiều nhất so với ảnh thật.
- Gradient tại biên chính là **tín hiệu** quyết định **densify** (thêm Gaussian mới) ở giai đoạn sau.
- Vùng gradient thấp $\Rightarrow$ Gaussian đã khớp tốt, không cần chia tách thêm.



Hình: Gradient tập trung ở biên Gaussian

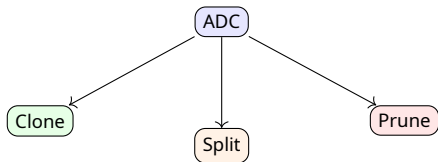
Tổng kết vòng lặp huấn luyện



- Một iteration: **render** → **loss** → **backward** → **optimizer.step()** → (định kỳ) **densify/prune** → lặp lại đến hội tụ.

Tổng quan Adaptive Density Control (ADC)

- Point cloud khởi tạo từ COLMAP thường **quá thừa** ở vùng thiếu chi tiết, đồng thời có Gaussian **quá to** che khuất chi tiết nhỏ.
- ADC chạy định kỳ mỗi `densification_interval` iteration để điều chỉnh mật độ Gaussian:
 - Densify**: nhân bản (clone) hoặc tách (split) Gaussian thiếu hình học.
 - Prune**: cắt tỉa Gaussian thừa, mờ, vô dụng.
- Mục tiêu: cân bằng chất lượng tái tạo và số lượng Gaussian (bộ nhớ, tốc độ render).



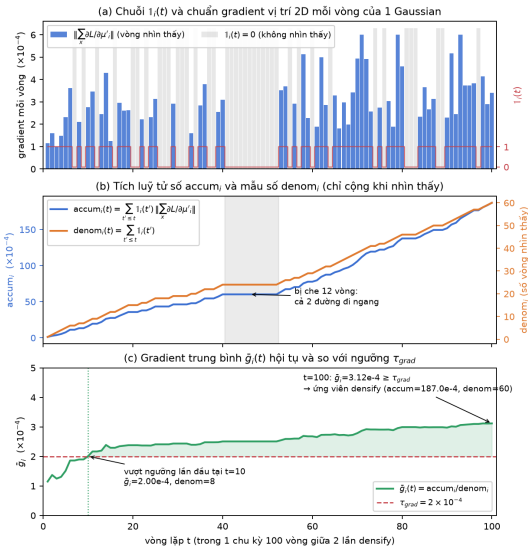
Tích lũy gradient view-space

- Với mỗi Gaussian, gradient vị trí 2D (view-space) được **cộng dồn** qua nhiều lần render từ các camera khác nhau.
- Đồng thời đếm $denom_i$: số lần Gaussian đó thực sự visible trong batch.

- Gradient trung bình dùng làm tín hiệu densify:

$$\bar{g} = \frac{\sum_v \nabla_{view}}{denom}$$

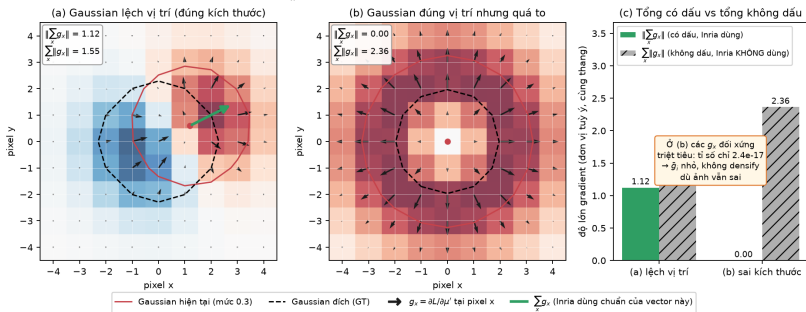
- $\bar{g}$ lớn $\Rightarrow$ ứng viên clone/split.



Vấn đề triệt tiêu gradient (gradient cancellation)

- Khi lấy trung bình vector gradient từ **những view có hướng ngược nhau**, chúng có thể **triệt tiêu lẫn nhau**.
- Hệ quả: dù mỗi view riêng lẻ báo lỗi lớn, $\bar{g}$ trung bình vẫn nhỏ $\Rightarrow$ Gaussian **bị bỏ sót**, không được densify.
- FastGS cải thiện bằng **multi-view consistency score** thay vì chỉ dựa vào gradient trung bình.

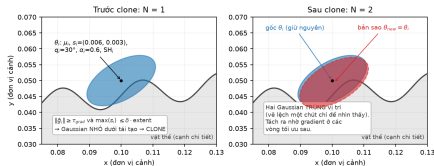
Inria chỉ tích lũy $\|\sum_x \partial L / \partial \mu'_{i|x}\|$: gradient pixel ngược chiều triệt tiêu trong kernel



Clone (nhân bản)

- Áp dụng cho Gaussian **nhỏ nhưng thiếu hình học** (under-reconstruction).
- Nhân đôi Gaussian theo **hướng gradient vị trí $\bar{g}$** , đặt bản sao lệch một khoảng nhỏ.
- **Không** thay đổi kích thước/covariance – chỉ lấp đầy vùng thiếu chi tiết bằng tăng mật độ.

Clone: sao chép y nguyên θ , scale giữ, góc giữ $\rightarrow N: 1 \rightarrow 2$



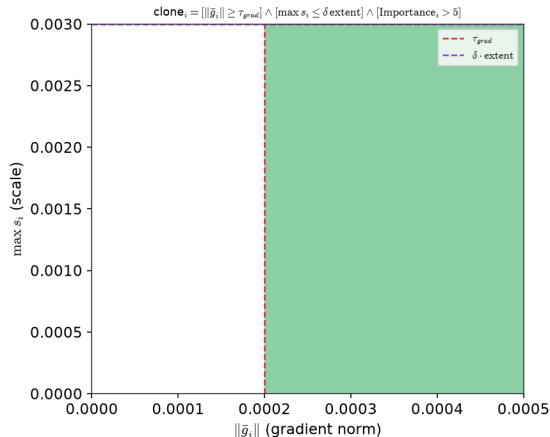
Bảng tham số: $\theta_{\text{new}} = \theta_i$

	góc θ_i	môđ θ_{new}
μ	(0.100, 0.050)	(0.100, 0.050)
s	(0.006, 0.003)	(0.006, 0.003)
q (góc)	30°	30°
α	0.6	0.6
SH	c_i	c_i

Mọi trường sao chép y nguyên: μ, s, q, α , hệ số SH. Không đổi scale, không xoay góc.

$N: 1 \rightarrow 2$

Hình: Trước/sau clone

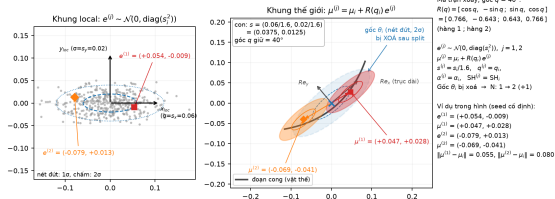


Hình: Vùng được clone

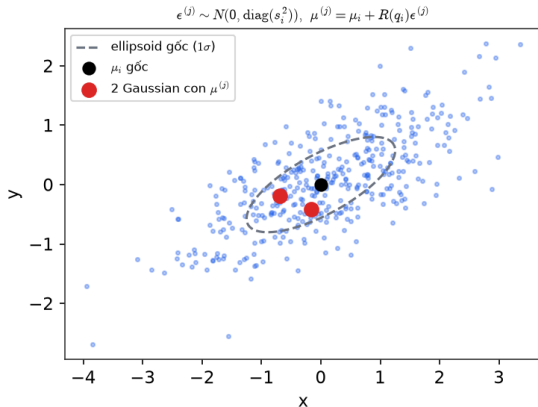
Split (tách)

- Áp dụng cho Gaussian **quá to** (over-reconstruction), vượt ngưỡng --dense theo scene extent.
- Tách thành **2 Gaussian con** nhỏ hơn (giảm scale theo hệ số cố định).
- Vị trí Gaussian con được **lấy mẫu ngẫu nhiên** theo phân phối chuẩn dựa trên covariance gốc của Gaussian cha.

Split: Gaussian TO ($\max(s_i) > \delta \cdot \text{extent}$) gán một đoạn cong $\rightarrow$ 2 con nhỏ hơn 1.6x



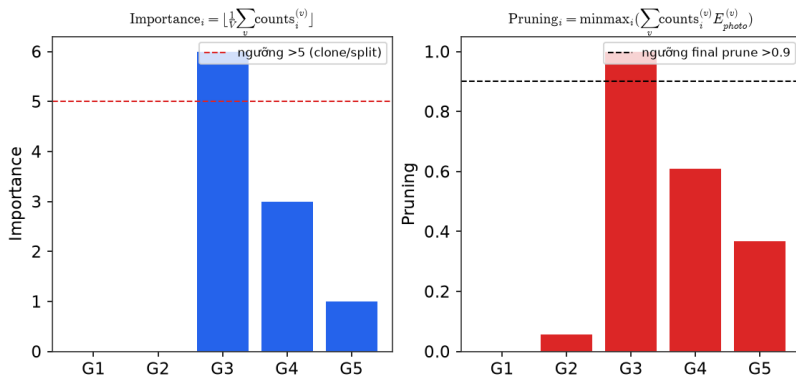
Hình: Hình học split



Hình: Lấy mẫu theo covariance

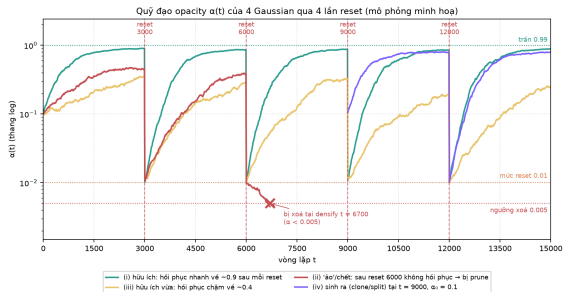
Prune theo độ quan trọng đa góc nhìn (FastGS)

- 3DGS gốc: prune đơn thuần theo ngưỡng **opacity thấp** hoặc **kích thước quá lớn**.
- **FastGS**: tính điểm *importance* cho mỗi Gaussian dựa trên đóng góp thực tế vào chất lượng ảnh render, tổng hợp qua **nhiều góc nhìn**.
- Gaussian có điểm importance thấp nhất bị cắt tỉa trước, bất kể opacity/kích thước.
- Kết quả: mô hình **gọn hơn** nhưng vẫn giữ chất lượng tái tạo – đây là **đòn bẩy chính** giúp FastGS tăng tốc.

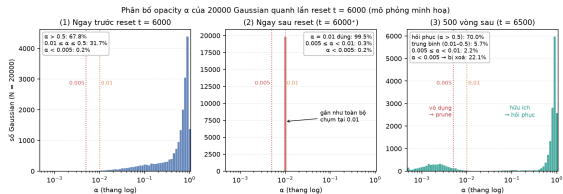


Reset opacity định kỳ

- Mỗi `opacity_reset_interval` (mặc định 3000 iteration): toàn bộ opacity reset về **giá trị thấp** (qua sigmoid-inverse).
- Buộc optimizer **"đánh giá lại"** từng Gaussian từ đầu.
- Gaussian thực sự cần thiết: opacity **tăng trở lại**; Gaussian dư thừa: không hồi phục kịp $\Rightarrow$ bị **prune** ở vòng sau.



Hình: Quỹ đạo opacity theo thời gian



Hình: Histogram trước/sau reset

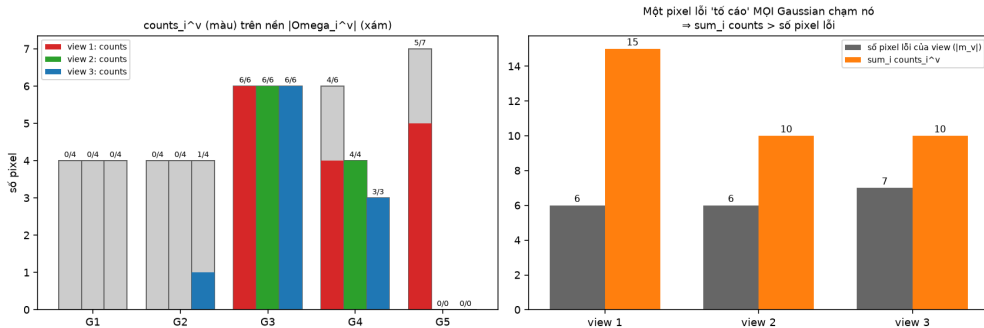
Tổng hợp counts qua nhiều view: đầu vào cho Importance

- Mỗi Gaussian i có vector counts ($\text{counts}_i^1, \text{counts}_i^2, \text{counts}_i^3$) theo từng view.
- Tổng số lỗi bị "tố cáo" trên mỗi view luôn $\geq$ số pixel lỗi thật:

$$\sum_i \text{counts}_i^v \geq |m_v|$$

vì một pixel lỗi có thể được nhiều Gaussian chồng lấn cùng đếm.

- Trong code: mỗi lần render view v , kernel CUDA cộng dồn atomic vào $\text{metricCount}[N]$, rồi lũy kế: $\text{full_metric_counts} += \text{counts}^{(v)}$.
- Vector counts tổng hợp qua V view là đầu vào trực tiếp cho bước tính Importance.



Hình: Trái: counts_i^v (màu) trên nền $|\Omega_i^v|$ (xám). Phải: $|m_v|$ so với $\sum_i \text{counts}_i^v$ mỗi view.

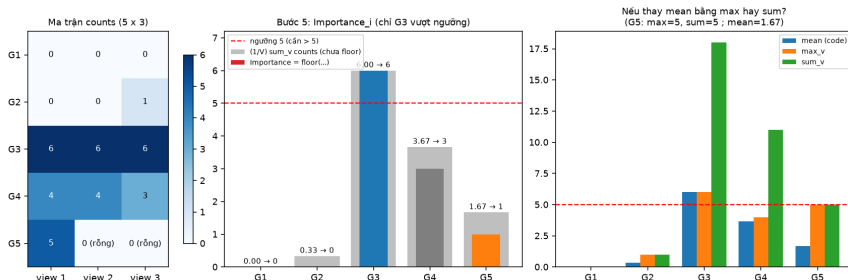
Importance score: trung bình counts qua các view, làm tròn xuống

- Công thức Importance (khớp code `compute_gaussian_score_fastgs`):

$$\text{Importance}_i = \left\lfloor \frac{1}{V} \sum_{v=1}^V \text{counts}_i^v \right\rfloor$$

`(torch.div(full_metric_counts, V, rounding_mode='floor'))`.

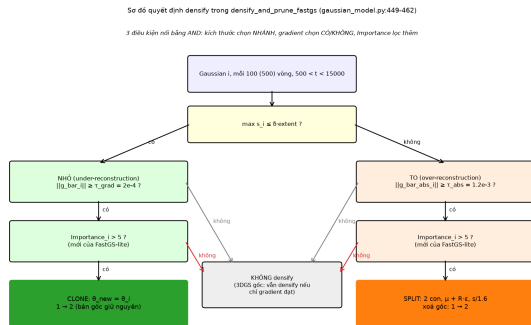
- Gaussian được xem là "quan trọng" (giữ lại/densify) khi $\text{Importance}_i > 5$.
- Ví dụ 4×4 , 3 view: chỉ G3 có mean > 5 vượt ngưỡng; các Gaussian còn lại bị chặn.
- Màu nóng trong heatmap = Gaussian đóng góp nhiều vào vùng lỗi cao (giữ lại); màu lạnh = ứng viên prune.



Hình: Trái: ma trận counts (5 x 3). Giữa: $\text{Importance}_i = \lfloor \text{mean}_V \rfloor$, ngưỡng 5. Phải: so sánh mean vs. max vs. sum.

Sơ đồ quyết định densify đầy đủ

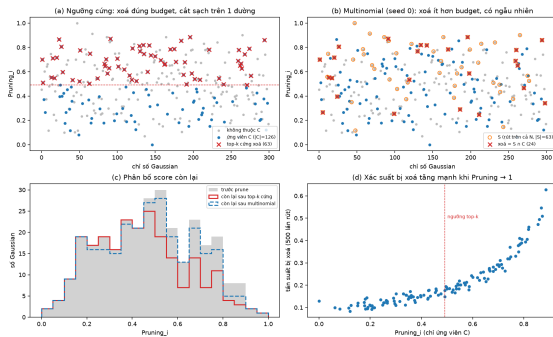
- Áp dụng cho mỗi Gaussian i , mỗi 100 (hoặc 500) vòng lặp, khi $500 < t < 15000$.
- **Bước 1 — kích thước chọn nhánh:** $\max s_i \leq \delta \cdot \text{extent}$?
- **Nhánh NHỎ:** nếu $\bar{g}_i \geq \tau_{\text{grad}} = 2 \times 10^{-4}$ **và** $\text{Imp}_i > 5 \Rightarrow$ **CLONE** ($1 \rightarrow 2$, giữ bản gốc).
- **Nhánh TO:** nếu $\bar{g}_{\text{abs},i} \geq \tau_{\text{abs}} = 1.2 \times 10^{-3}$ **và** $\text{Imp}_i > 5 \Rightarrow$ **SPLIT** (2 con, xoá gốc, $1 \rightarrow 2$).
- Nếu Importance không thỏa: 3DGS gốc vẫn densify, FastGS-lite thì không.
- Nguồn: `densify_and_prune_fastgs`, `gaussian_model.py:449-462`.



Hình: 3 điều kiện nối bằng AND: kích thước chọn nhánh, gradient chọn có/không, Importance lọc thêm.

Top-k cứng vs. Multinomial: hai chiến lược pruning

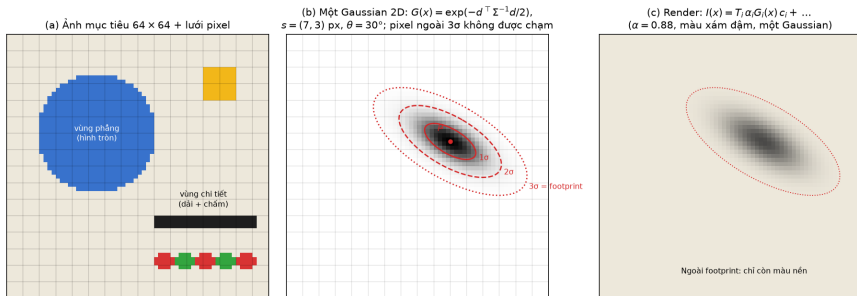
- Mô phỏng $N = 300$ Gaussian, điểm số P_i chuẩn hoá min-max từ $\text{Beta}(2,2,5)$; tập ứng viên C chọn ngẫu nhiên 40% quần thể, $\text{budget} = \lfloor 0,5|C| \rfloor$.
- **Top-k cứng:** xoá đúng budget phần tử điểm thấp nhất.
 - Cắt theo ngưỡng cứng – có thể xoá nhầm cả một cụm liên kề điểm số tương tự.
- **Multinomial (stochastic):** rút mẫu không hoàn lại theo $w_i \propto 1/(10^{-6} + 1 - P_i)$.
 - + Đa dạng hơn, tránh xoá đồng loạt một vùng.
 - Số lượng xoá thực tế thường *ít hơn* budget.



Hình: (a) top-k cứng cắt theo một đường ngưỡng; (b) multinomial xoá ít hơn budget, có ngẫu nhiên; (c) phân bố điểm còn lại; (d) xác suất bị xoá tăng mạnh khi $P_i \rightarrow 1$.

Toy model 2D: mục tiêu và một Gaussian

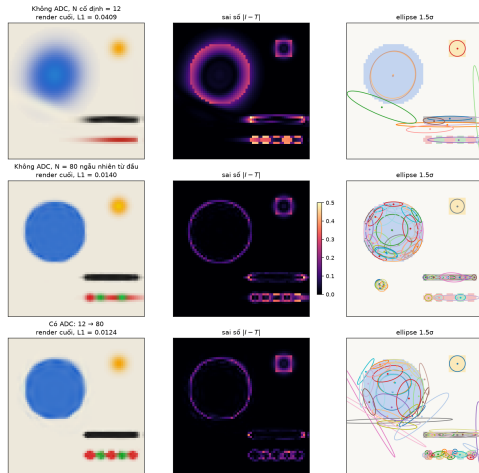
- Ảnh mục tiêu 64×64 px: hình tròn phẳng + vùng chi tiết (dải mảnh, chấm màu xen kẽ, ô vuông nhỏ).
- Mỗi Gaussian 2D: $G(x) = \exp(-\frac{1}{2}d^T \Sigma^{-1}d)$, footprint giới hạn ở 3σ .
- Render bằng alpha-compositing theo thứ tự chỉ số: $I(x) = \sum_i T_i \alpha_i G_i(x) c_i + T_{\text{end}} \cdot \text{bg}$, với $T_i = \prod_{j < i} (1 - \alpha_j G_j)$.



Hình: (a) Ảnh mục tiêu + lưới pixel; (b) một Gaussian $s = (7, 3)$ px, $\theta = 30^\circ$ với các đường mức 1σ , 2σ , 3σ ; (c) kết quả render của Gaussian đó lên nền.

So sánh trực tiếp: không ADC vs có ADC

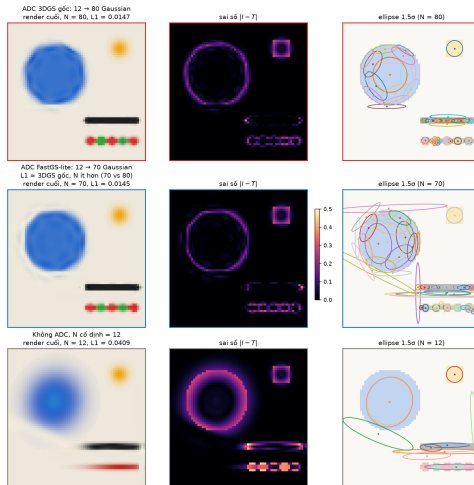
- Ba kịch bản: (1) không ADC, $N=N_0$; (2) không ADC, khởi tạo ngẫu nhiên $N=N_{\text{cuối}}$; (3) có ADC, $N_0 \rightarrow N_{\text{cuối}}$ dần.
 - (2) cùng số Gaussian (3) nhưng khởi tạo ngẫu nhiên không khớp tự nhiên vào vùng chi tiết.
- ⇒ ADC không chỉ tăng N mà đặt đúng chỗ cần thiết.



Hình: 3 hàng (không ADC N_0 ; ngẫu nhiên; có ADC) $\times$ 3 cột.

Kết quả render cuối cùng: 3 kịch bản đối chiếu

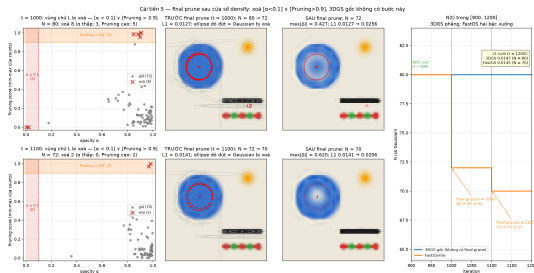
- Hàng 1 — **ADC 3DGS gốc**: N tăng từ khởi tạo lên N cuối, L_1 cuối như trong hình.
 - Hàng 2 — **ADC FastGS-lite**: L_1 cuối **tương đương** nhưng **ít Gaussian hơn** — hiệu quả của Importance + prune multinomial + final prune.
 - Hàng 3 — **Không ADC**: $N=N_0$ cố định, không clone/split/prune — chất lượng kém hơn rõ rệt.
- Mỗi hàng: render cuối, sai số $|I - T|$, ellipse 1.5σ . Viền màu: đỏ=3DGS, xanh=FastGS, xám=không ADC.



Hình: Đối chiếu cuối: ADC 3DGS gốc / ADC FastGS-lite / không ADC.

Tổng kết 5 cải tiến ADC của FastGS-lite

- **5 cải tiến ADC:** (1) Importance đa view AND với gradient; (2) split theo gradient tuyệt đối; (3) prune multinomial; (4) trần opacity 0.8; (5) final prune sau densify.
 - Final prune tại $t \in (1000, 1100)$: $rm = [\alpha < 0.1] \vee [\text{Pruning} > 0.9]$.
 - 3DGS gốc không có bước này - $N(t)$ phẳng [900, 1200]; FastGS giảm "hai bậc thang".
 - Scatter (α , Pruning): vùng "chữ L" bị xoá, đánh dấu X đỏ.
- ⇒ **Kết luận:** L_1 tương đương 3DGS gốc nhưng N nhỏ hơn hẳn - khép lại chương ADC, mở sang phần tăng tốc.



Hình: Final prune sau densify: xoá $[\alpha < 0.1] \vee [\text{Pruning} > 0.9]$.

Vì sao 3DGS gốc chậm?

- Mỗi iteration gồm: **render** (rasterize), **tính loss**, **backward**, định kỳ **densify/prune**.
- Với hàng trăm nghìn – hàng triệu Gaussian, **rasterize** và **backward** chiếm phần lớn thời gian mỗi bước.

Khâu	Tần suất	Tỉ trọng thời gian
Render (project + rasterize)	mỗi iteration	~45%
Backward (gradient)	mỗi iteration	~35%
Tính loss	mỗi iteration	~10%
Densify/Prune	định kỳ	~10% (đột biến)

Ba đòn bẩy tăng tốc của FastGS (tổng quan)

1. Multi-view score

- Đánh giá Gaussian qua nhiều góc nhìn
- Pruning thông minh hơn

2. Compact Box

- Giảm hệ số nhân bounding-box
- Giảm số tile chạm

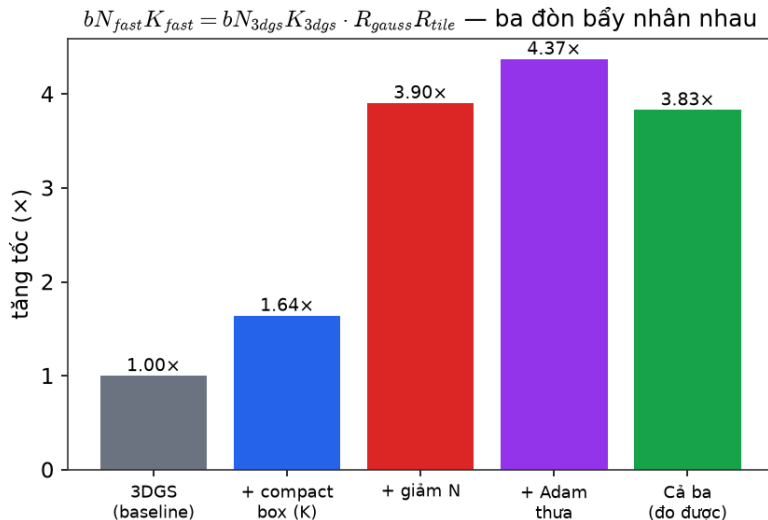
3. Sparse Adam

- Lịch cập nhật phân tầng
- Giảm tần suất optimizer.step()

- Cả ba đòn bẩy nhằm đúng khâu chiếm tỉ trọng lớn nhất: rasterize, densify/prune, cập nhật tham số.

Hiệu năng tích lũy: Biểu đồ thác nước

- Waterfall chart minh họa tăng tốc **tích lũy** khi bật lần lượt từng đòn bẩy lên baseline 3DGS gốc.
- Thứ tự: baseline → + Compact Box → + Multi-view score → + Sparse Adam → FastGS đầy đủ.



Tổng kết: từ 3DGS gốc đến FastGS

Tham số	3DGS gốc	FastGS
loss_thresh	không có	mới, 0.1
grad_abs_thresh	gradient thường	Abs-GS, 0.0012
--dense	cố định	0.001–0.013 (theo scene extent)
--highfeature_lr	lr chuẩn	0.005–0.02 (chia thêm /20)
--mult (box multiplier)	3.0	0.5–0.7

- FastGS không đổi mô hình biểu diễn cảnh, mà tối ưu **tham số hoá** và **lịch huấn luyện** dựa trên mô hình chi phí và định luật Amdahl $\Rightarrow$ tăng tốc mà PSNR/SSIM gần như không đổi.

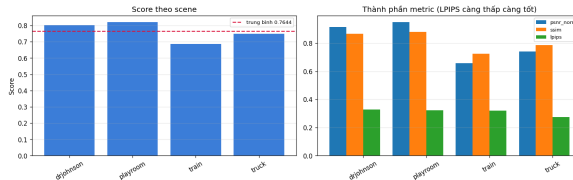
- **fastgs-lite**: fork giữ nguyên thuật toán FastGS gốc, chỉ đóng gói lại để chạy trên Colab T4 free tier – không đổi công thức toán hay pipeline huấn luyện.

Tiêu chí	FastGS gốc	fastgs-lite
Phần cứng mục tiêu	24GB RTX 4090	T4 (~15GB VRAM) + 12.7GB RAM
CUDA extension	git submodule	vendor sẵn trong repo (<code>git clone</code>)
Entry point	train.py + shell script	train.py + notebook có instrumentation
Tín hiệu tiến trình	L1 + EMA	Score tổng hợp, log mỗi 1000 iter

Bảng 1: Chất lượng cuối cùng theo scene

Scene	Score	PSNR (dB)	SSIM	LPIPS	#Gaussian
playroom	0.8198	-	-	-	~129k
drjohnson	0.8027	27.41	-	-	~174k
truck	0.7482	-	-	-	~187k
train	0.6867	19.75	-	-	~201k
Mean	0.7643	24.46			

- 2 scene indoor vượt 2 scene outdoor dù dùng ít **Gaussian hơn** (129k-174k so với 187k-201k).



Hình: Leaderboard Score theo scene

Bảng 3: Chi phí & lưu trữ

- Tổng train 4 scene: **327.6s** (~5.5 phút), **85.5 iter/s** trung bình, VRAM đỉnh chỉ **0.88GB** (max 1.16GB).
- .ply trung bình **171.6MB**, đúng **248 byte/Gaussian** = record 3DGS không nén.

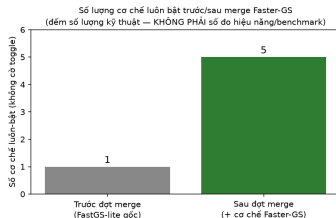
Chỉ số	Trung bình	Ghi chú
Thời gian train (4 scene)	327.6 s	~5.5 phút tổng
Tốc độ	85.5 iter/s	
VRAM peak	0.88 GB	max 1.16GB (drjohnson)
Kích thước .ply	171.6 MB	248 byte/Gaussian

```
Config(resolution=2, iterations=7000, score_every=1000,  
       eval_views=6, psnr_max=30.0, submission_resolution=1)
```

- **Giới hạn:** chạy **một lần duy nhất** (không seed/lặp lại), chỉ **4 scene** từ 2 dataset, ngân sách 7k iteration chỉ bằng **1/3** lịch 30k thiết kế gốc, bỏ qua bước prune cuối (`final_prune_fastgs`).
- ⇒ **Kết luận:** Bảng 1 là *baseline có thể tái lập của repo này*, **không phải** benchmark chính thức để so sánh với FastGS/3DGS gốc.

Faster-GS (CVPR 2026): 4 cơ chế mới luôn bật

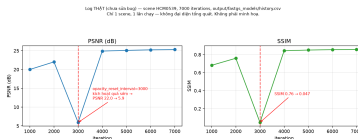
- **Fused Adam**: 1 kernel CUDA elementwise, đúng công thức Adam chuẩn (không đổi thuật toán, chỉ giảm overhead kernel-launch).
 - **3D anti-aliasing filter**: clamp scale tối thiểu theo khoảng cách camera $\Rightarrow$ chống alias.
 - **Morton Z-order reordering**: sắp Gaussian theo mã bit-interleave (mỗi 5000 iteration) $\Rightarrow$ tăng cache locality cho rasterizer.
 - **Random-init fallback**: sample đều trong hình cầu bằng $r = R \cdot u^{1/3}$ khi COLMAP thiếu `points3D`.
- $\Rightarrow$ Cả 4 đều **luôn bật**, không còn cờ toggle.
- *Ngoại lệ*: MCMC densification có sẵn trong code nhưng **giữ tắt** — xung đột kiến trúc với densify chính của FastGS.



Hình: Số cơ chế luôn bật trước/sau đợt merge (đếm số lượng, không phải benchmark)

Bug đã sửa: opacity_reset_interval không co giãn theo iterations

- **Nguyên nhân thật** của các điểm sụt Score đã thấy (Slide 33–37): `opacity_reset_interval` hardcoded **3000** (đúng cho lịch gốc 30k), **không co giãn** theo `iterations` đã rút ngắn $\Rightarrow$ reset kích hoạt quá sớm.
- Bằng chứng thật: PSNR rơi **22.02** $\rightarrow$ **5.92** tại iteration 3000 (scene HCM0539).
- **Cách sửa**: field mới `opacity_reset_frac=0.2`, co giãn `opacity_reset_interval = round(0.2  $\times$  iterations)`.
- Chưa có log thật "sau khi sửa" — chưa build/chạy lại (không có GPU ở môi trường phát triển).



Hình: Log thật (trước khi sửa): PSNR sụt tại iteration 3000

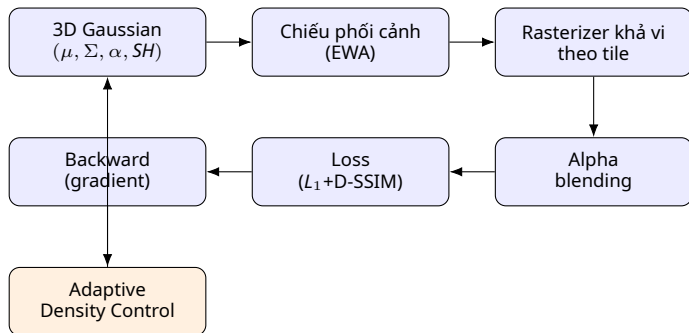
Quan sát 2: Nút thắt là RAM hệ thống, không phải VRAM

- Đo trên Google Colab T4 trong quá trình huấn luyện:

Tài nguyên	Đỉnh sử dụng	Khả dụng	% sử dụng
VRAM (GPU)	1.16 GB	14.56 GB	8%
RAM hệ thống	9.59 GB	12.7 GB	91% (giới hạn mềm 10.5 GB)

- RAM hệ thống áp sát giới hạn mềm, trong khi VRAM còn dư rất nhiều.
- Khuyến nghị cũ "giảm độ phân giải để tiết kiệm VRAM" **không cần thiết** ở ngân sách này.

Tổng kết pipeline render 3D Gaussian Splatting



Một chu trình khép kín: render → tính loss → lan truyền ngược → điều chỉnh mật độ Gaussian → lặp lại.

Ba đòn bẩy chính:

- **Multi-view score + Compact box** $\Rightarrow$ giảm Gaussian dư thừa mà không giảm chất lượng đáng kể.
- **Sparse Adam** $\Rightarrow$ giảm chi phí tính toán mỗi iteration.

Giới hạn / hướng phát triển:

- ❶ Dữ liệu Gaussian đầu ra chưa được nén (248 B/Gaussian, thô).
- ❷ Ngân sách 7k iteration bỏ lỡ bước `final_prune`.
- ❸ Chưa có ablation study cô lập riêng từng đòn bẩy.
- ❹ 4 nhánh mở rộng của FastGS gốc — dynamic scenes, sparse view, surface reconstruction, SLAM — chưa tích hợp vào fork này.

Cảm ơn đã theo dõi!

Hỏi đáp (Q&A)

Digital Twin GS — FastGS-lite
github.com/KietAnhCS/fastgs-lite